

1. Liquidity formula might not match the proportion we want.

2. the selected call-put pair for each secid
was ranked by ["combined spread", "entry-mid"].

In this selection, we neglected "open interests."

- Define a function that gathers all of the third Fridays that we need (2018/01/01 to 2023/12/31). F_t collects all third Fridays and E_1 collects all third Fridays for the next month.

- Import the option price folder, stock permno with secid folder, and the stock price folder

- For a specific option day, we extract:

1. date 2. expiration date 3. mid = $\frac{\text{best bid} + \text{best offer}}{2}$ 4. $k = \frac{\text{strike price}}{1000}$

then we have output df ^(dataframe) table with all info for that specific day's option. = strike price in dollars

- set up load_stock function that gives a stock dataframe with columns: "date", "permno", "prc" all numeric, and column "s" taking the absolute value of "prc".

- set up attach_stock_price_at_expiry for future usage

get a copy of the dataframe: "selected"

match E_t1 with stock_date, requiring permno to be the same from dataframe "out" and "stock_small" (this is stock DataFrame with only "permno", "date", "s" columns).

When we are matching, we use the latest stock_date $\leq E_t1$

- The BIG Function: choose the top 20 secids for a specific date.

General filtering logic:

1. Determine the month, there will be one corresponding option date F_t (the third Friday) and one corresponding expiration date E_t1 (third Friday of next month)
2. For the options on day F_t , filter out non-standard ones, and filter out those that dates $\neq E_t1$.
3. Drop off the rows without important data.

4. require $\text{best_offer} \geq \text{best_bid}$, $\text{best_offer} > 0$, $\text{best_bid} > 0$, and $\text{mid} > 0$.

★ 5. define the liquidity formula : $\text{liq_score} = \frac{\text{total open interest}}{1 + \text{median spread}}$
and rank the secids with their liq-scores. Get the top 20 secids after ensuring $\text{sdate} \leq F_t \leq \text{edate}$ and break ties by most recent sdate and save these secids into a list.

6. For the big dataframe we initially have for that specific date, filter to leave only those secids in the top 20 list.

7. Then, we start to build straddles

calls (a DataFrame) such that :

1. $\text{CP_flag} == "C"$ 2. $"\text{open_interest}" > 0$ 3. $"\text{delta}" \in [0.25, 0.75]$.

puts (a DataFrame) such that

1. $\text{CP_flag} == "P"$ 2. $"\text{open_interest}" > 0$

8. Create straddle pairs:

pair calls with puts that have

- 1) same secid
- 2) same expiration date
- 3) same strike price in dollars K .

9. Add $\text{combined_spread} = \text{call_spread} + \text{put_spread}$

10. $\text{weight for call (wC)} = \frac{-\text{put_delta}}{\text{call_delta} - \text{put_delta}}$

$\text{weight for put (wP)} = 1 - \text{weight for call}$

filter to leave with $wC \in [0, 1]$ and $wP \in [0, 1]$

11. For each delta-neutral straddle, calculate

"entry_mid" = $wC \times \text{call_mid} + wP \times \text{put_mid}$

12. for each secid, get the one pair with smallest combined spread and break ties with smallest entry mid.

Now, for each of the 20 secids, we have the best straddles for each, saved in a DataFrame called "selected"

13. Then, we use the previously set up function `attach_stock_price_at_expiry`.

$$\text{payoff}_C = \max(S - K, 0)$$

$$\text{payoff}_P = \max(K - S, 0)$$

The exit value of the delta-neutral straddle:

$$\text{"exit_intrinsic"} = w_C \times \text{payoff}_C + w_P \times \text{payoff}_P$$

Make sure $\text{entry_mid} > 0$, $S > 0$, $K > 0$.

14. $R_{t1} = \text{straddle return} = \frac{\text{exit_intrinsic} - \text{entry_mid}}{\text{entry_mid}}$

The big function returns the monthly selected straddles, realized return and other key columns. Organized everything into "selected" DataFrame

Then, Run over all months and then stack all monthly DataFrames into one big panel dataset across months.

The overall big DataFrame is "all-monthly".

Step 5:

For each permno, we take its time series of R_{t1} and use `shift(2)` to capture the last 11 shifted months, the return for months $t-12, t-11, \dots, t-2$. And the average all of them to get $\text{Mom}_{\{t\}}$.

Note that if < 8 non-NaN values in the 11 months we captured, Mom_{rt} becomes NaN.

"steps" is a DataFrame copied from all_monthly.

and we give an extra column [Mom_rt] so that each permno would have the ^{past} return available shown up for each month.

- produce the mom_table
- mom_table stats table Overview
- the heatmap plot (only the top 60 permnos with the most MoM observation)

Step 6:

- We set the number of quantile buckets to 5.

Let panel 6 be steps[["month", "permno", "R_t1", "Mom_rt"]]

We should sort "permno" and "month" and obtain "R_next" at month "R_t1" using shift(-1).

Also we automatically drop lines that miss any of the above 4 elements, which drop the initial 7 months for all permnos and the very last month because it does not have R_next.

★ Note that in this step we use winsorization, meaning that although we keep all data as they are, we set CAP_LO = -0.95 and CAP_HI to avoid Mom_rt of any permno to be strongly dominated by single extreme month. If the return for some month of a permno is less than -95% or +500%, we treat them as these two extremes as a modeling guardrail to get a better sense of normal situation rather than being entirely skewed by an outlier.

panel6["R_next_cap"] = panel6["R_next"] after limiting to CAP_LO and CAP_HI.

Define function bucketize_mom to categorize stocks into buckets, not enough then put NA.

Step 7:

set up function eval_series: (input pandas ^{time-series} $\checkmark$) (output dictionary that shows all below info)

- Sample mean
- Sample standard deviation
- annualized return = $12 \times \text{month mean}$
- annualized volatility = $\text{sqrt}(12) \times \text{month volatility}$
- annualized sharpe = $\text{annualized return} / \text{annualized volatility}$
- monthly sharpe = $\frac{\text{month mean}}{\text{month std}}$
- tstat = $\frac{\text{month mean}}{\text{month vol} / \text{sqrt}(\# \text{ months})}$
- Also check whether can compound, check if any month has monthly return < -1 .
we check it because it would mean we theoretically cannot continue at that point and we have no more fund.

Set up another function plot_cumulative with x-axis months and y-axis cumulative (additive sum).

1. First plots a PnL-like line graph for additive cumulative sum.
x-axis: Month y-axis: cumulative sum,

2. If it can compound, we will calculate its cumulative return by the formula: $\prod (1 + R_t) - 1$, we can also plot it with y-axis cumulative return and x-axis month

set up function plot_hist to visualize count of monthly returns for each bin. x-axis: monthly return, y-axis: Count.

- eval_series (LS_5050) } $\Rightarrow$ summary
- eval_series (LS_spread) }
- plot_cumulative (LS_5050)
- plot_hist (LS_5050)
- plot_cumulative (LS_spread)
- plot_hist (LS_spread)

step 8 :

same steps 5-7.

Take the return series for each permno and apply the same rolling method, past 11 months, requiring 8 minimum, calculate the sample standard deviation (risk estimate). (variable name: sigma_rt)

Change the weighting using the following formula:

The inverse volatility weighting:

$$w_raw = \frac{1}{\sigma_{rt} + 1 \times 10^{-8}}$$

Then normalize weighting in each group.

$$w_norm = \frac{w_raw}{p.groupby(["month", "bucket"])["w_raw"].transform("sum")}$$

group by signal month and bucket, similarly graph.

step 9 add slippage rate entry, exit cost rate, cost base points per leg,

$$\# \text{ legs}, \quad \text{cost_rate_total} = \frac{\text{cost_bps_per_leg}}{10000} \times \# \text{ legs}.$$

$$\text{net return} = \frac{\text{entry_mid} \times (1 + \text{slippage_rate_entry}) - \text{exit_intrinsic} \times (1 - \text{exit_cost_rate}) - \text{entry_mid} \times \text{cost_rate_total}}{\text{entry_mid}}$$

then repeat step 5 - step 8 and graph

Tasks

- Add net performance summary to step 8 of strategy 1.
- Add steps 8-9 to strategy 2 and 3